Full Stack Development with MERN

Project Documentation

1. Introduction

- **Project Title:** HotelPro – Hotel Room Booking and Management System
- **Team Member ID:** SWUID2026402745

Team Members:

Team Member ID	Name	Role
SWUID2026402745	Ronak Sahu	Software Developer

2. Project Overview

- **Purpose:** HotelPro is a web-based hotel room booking and management system. It allows customers to search for hotels and rooms, check availability, book rooms and pay online, while hotel staff and administrators manage hotels, rooms, guests, bookings, check-in and check-out, and payments from a single platform.

The main goals of the project are to:

- provide a simple, fast and secure online platform for hotel room reservation;
- eliminate double bookings by maintaining accurate, real-time room availability;
- reduce manual paperwork for hotel staff through digital guest and booking management;
- offer secure, role-based access for customers, staff and administrators;
- give administrators useful insights through a dashboard of bookings, occupancy and revenue; and
- deliver a modular, scalable and maintainable code base that can be extended with future features.

Features: The key features and functionalities of HotelPro are as follows.

Module	Key Functionality
User Registration and Login	Account creation with validated input; secure login with email and password; password hashing with bcrypt; profile view and update; logout.
JWT-Based Authentication	Signed JSON Web Tokens issued on login; token verification and role-based authorisation on every protected request.
Hotel and Room Management	Administrators add, edit and delete hotels and rooms; room types (Single, Double, Deluxe, Suite); room status (available, occupied, maintenance, inactive).
Room Search and Availability	Search by city/hotel, check-in and check-out dates and number of guests; filter and sort by price, room type, amenities and rating; only available rooms are shown.
Hotel Room Booking	Booking for selected dates; total price calculation including taxes; availability re-verified at confirmation; unique booking ID with initial status “Pending”.
Booking Management	Customers view, modify and cancel bookings; staff and administrators search and filter all bookings; statuses: Pending, Confirmed, Checked-in, Checked-out, Cancelled.
Guest Management	Guest records (name, contact, address, ID proof) linked to bookings; additional guests; protected access to personal data.
Check-in and Check-out	Check-in against a confirmed booking; room status updated automatically; actual check-in and check-out times recorded.

Module	Key Functionality
Payment Processing	Online payment through an integrated payment gateway (card, UPI, net banking); booking confirmed only after successful payment; payment records, receipts and refunds; offline payments recorded by staff.
Admin Dashboard	Key metrics (total and active bookings, cancellations, occupancy rate, available rooms, revenue); user management; booking and payment views; report export.

3. Architecture

HotelPro follows a three-tier client–server architecture. The React.js client communicates with the Node.js and Express.js server over HTTPS using JSON-based REST APIs. The server implements the business logic, authenticates requests with JSON Web Tokens and stores data in MongoDB through Mongoose. The server also integrates with an external payment gateway.

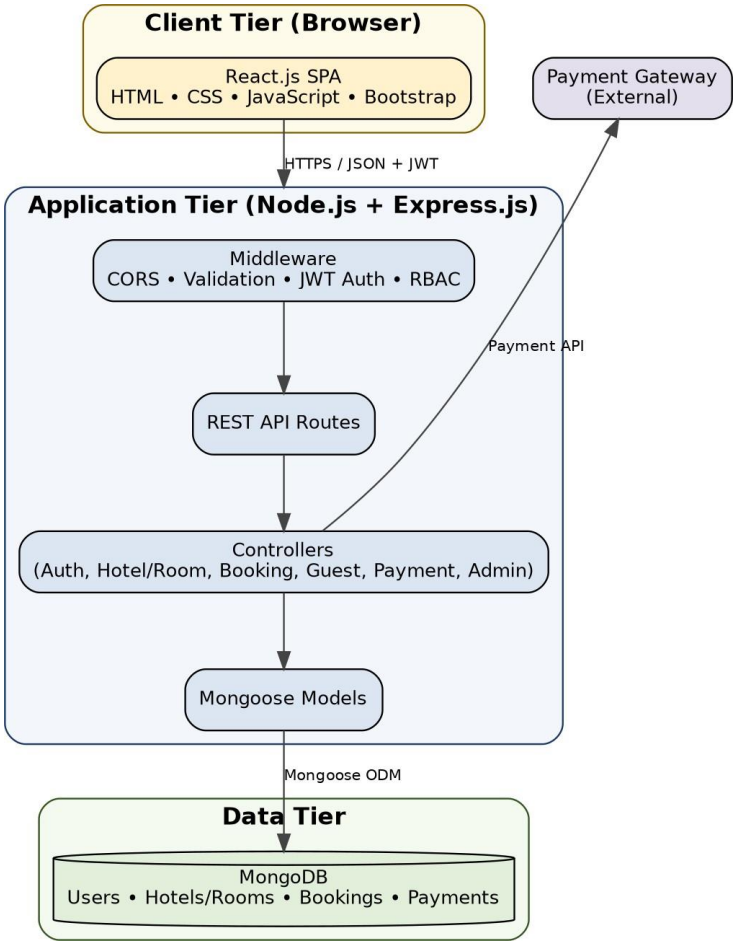


Figure 1 – System architecture of HotelPro

Frontend

- **Technology:** a responsive single-page application (SPA) built with React.js, HTML, CSS, JavaScript and Bootstrap.
- **Routing and state:** React Router is used for navigation; protected routes redirect unauthenticated users to the login page, and the JWT is attached to protected API requests (Axios).
- **Principal screens:** Home / Landing, Register / Login, Search Results, Room Details, Bookings.

- **General UI rules:** consistent navigation bar and footer, uniform colour theme, visible error and success notifications, loading indicators for asynchronous operations and inline form validation.

Backend

- **Technology:** Node.js runtime with the Express.js framework, exposing a RESTful API that exchanges data in JSON format.
- **Layers:** middleware (CORS, input validation, JWT authentication and role-based authorisation) → routes → controllers (authentication, hotel/room, booking, guest, payment and admin) → Mongoose models.
- **Responsibilities:** business logic such as availability checking, price calculation, booking status transitions and payment recording; standard HTTP status codes (200, 201, 400, 401, 403, 404, 409, 500) are used.
- **External integration:** a third-party payment gateway (sandbox for testing, live for production). Card data is never stored by HotelPro.

Database

HotelPro uses MongoDB, a document-oriented NoSQL database. Data is organised into the collections below; Mongoose schemas enforce structure and validation, and documents reference each other by ObjectId.

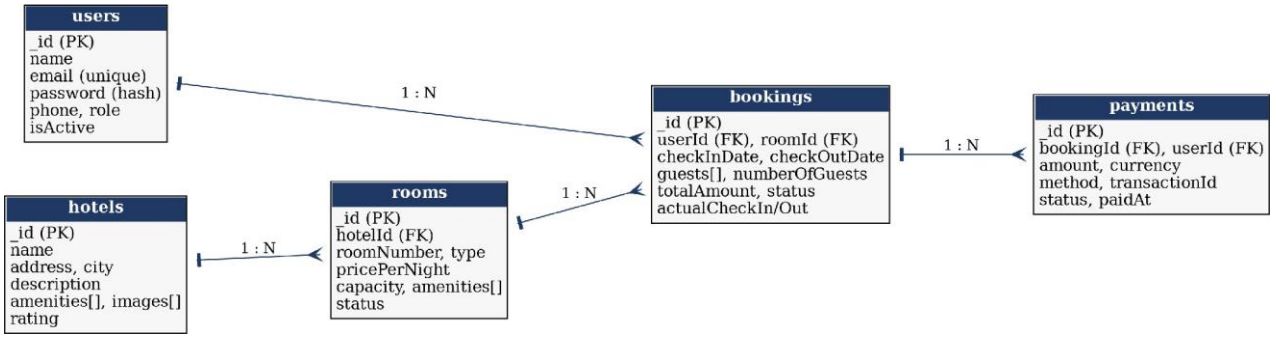


Figure 2 – Entity relationship diagram of HotelPro

users – stores account and authentication data for all roles.

Field	Type	Constraints / Description
`_id`	ObjectId	Primary key, auto-generated
`name`	String	Required, full name
`email`	String	Required, unique, lowercase, validated format
`password`	String	Required, bcrypt hash (never returned in API responses)
`phone`	String	Optional, contact number
`role`	String	Enum: customer, staff, admin; default customer
`isActive`	Boolean	Account status; default true
`createdAt / updatedAt`	Date	Timestamps (automatic)

hotels – describes each hotel property.

Field	Type	Constraints / Description
`_id`	ObjectId	Primary key
`name`	String	Required

Field	Type	Constraints / Description
address / city	String	Required; city indexed for search
description	String	Optional
amenities	Array[String]	For example, WiFi, Parking, Pool
images	Array[String]	Image URLs
rating	Number	Average rating, 0–5

rooms – each room references its hotel.

Field	Type	Constraints / Description
_id	ObjectId	Primary key
hotelId	ObjectId	Reference to hotels._id (required)
roomNumber	String	Required; unique within a hotel
type	String	Enum: single, double, deluxe, suite
pricePerNight	Number	Required, greater than 0
capacity	Number	Maximum guests, required
amenities	Array[String]	Room-level amenities
status	String	Enum: available, occupied, maintenance, inactive

bookings – stores reservations and guest details.

Field	Type	Constraints / Description
_id	ObjectId	Primary key; also used to derive the booking reference
userId	ObjectId	Reference to users._id (required)
roomId	ObjectId	Reference to rooms._id (required)
checkInDate	Date	Required
checkOutDate	Date	Required; must be after checkInDate
guests	Array[Object]	Guest name, contact, ID proof type and number, age
numberOfGuests	Number	Adults + children
totalAmount	Number	Calculated total including taxes
status	String	Enum: pending, confirmed, checked-in, checked-out, cancelled
actualCheckIn / actualCheckOut	Date	Recorded at check-in / check-out
createdAt / updatedAt	Date	Timestamps (automatic)

payments – stores transaction records; no sensitive card data is stored.

Field	Type	Constraints / Description
_id	ObjectId	Primary key
bookingId	ObjectId	Reference to bookings._id (required)
userId	ObjectId	Reference to users._id (required)

amount	Number	Required, greater than 0
currency	String	Default INR

Field	Type	Constraints / Description
method	String	Enum: card, upi, netbanking, cash
transactionId	String	Gateway reference; unique
status	String	Enum: pending, success, failed, refunded
paidAt	Date	Timestamp of successful payment

Indexes: unique index on users.email; compound index on bookings (roomId, checkInDate, checkOutDate) for availability queries; index on hotels.city; unique index on payments.transactionId.

Interactions: one hotel has many rooms; one user makes many bookings; one room can appear in many bookings over time; and one booking can have one or more payment transactions (for example, a failed attempt followed by a successful one).

4. Setup Instructions

Prerequisites

- Node.js (LTS version) and npm
- MongoDB 6.0 or later (local installation) or a MongoDB Atlas cluster
- Git, and a GitHub account for the repository
- A modern web browser (latest version of Chrome, Firefox, Edge or Safari)
- Visual Studio Code (code editor) and Postman (API testing)
- Operating system: Windows 10/11, Linux (Ubuntu 20.04 or later) or macOS

Installation

Follow these steps to clone the project, install the dependencies and configure the environment variables.

Step 1 – Clone the repository

```
git clone <your-repository-url>
cd <project-folder>
```

Step 2 – Install the backend dependencies

```
cd server
npm install
```

Step 3 – Install the frontend dependencies

```
cd ../client
npm install
```

Step 4 – Configure the environment variables. Create a `.env` file in the `server` directory. As required by the SRS, the JWT secret and the database credentials are kept in environment variables and are never committed to the repository. The variable names below are examples.

```
PORT=5000
MONGO_URI=<your-mongodb-connection-string>
JWT_SECRET=<your-secret-key>
JWT_EXPIRES_IN=1h
PAYMENT_GATEWAY_KEY=<your-payment-gateway-sandbox-key>
```

Step 5 – Make sure MongoDB is running (local service) or that the Atlas connection string.

5. Folder Structure

The SRS requires a modular code base (routes, controllers, models, middleware and services). The structure below follows that guideline.

Client

```
client/
|-- public/                 static files and index.html
|-- src/
|   |-- components/         reusable UI (Navbar, Footer, RoomCard, SearchBar, forms)
|   |-- pages/              Home, Login, Register, SearchResults, RoomDetails,
|   |                       Booking, MyBookings, StaffConsole, AdminDashboard
|   |-- routes/             protected / role-based route wrappers
|   |-- context/            authentication state (user, token, role)
|   |-- services/           Axios API calls to the backend
|   |-- App.js              route definitions
|   `-- index.js            application entry point
`-- package.json
```

Server

```
server/
|-- config/                 database connection and configuration
|-- controllers/            authController, hotelController, roomController, bookingController,
|                           guestController, paymentController, adminController
|-- middleware/             authentication (JWT), authorisation (roles), validation, error handling
|-- models/                 User, Hotel, Room, Booking, Payment (Mongoose schemas)
|-- routes/                 authRoutes, hotelRoutes, roomRoutes, bookingRoutes,
|                           guestRoutes, paymentRoutes, adminRoutes
|-- services/               availability check, price calculation, payment gateway integration
|-- .env                    environment variables (not committed to Git)
|-- server.js               application entry point
`-- package.json
```

The Client and Server structures show the recommended organisation. Please adjust the folder and file names if they differ in your repository.

6. Running the Application

Start the backend and the frontend servers locally in two separate terminals.

- **Backend:** `npm start` in the `server` directory.
- **Frontend:** `npm start` in the `client` directory.

```
# Terminal 1 - backend
cd server
npm start

# Terminal 2 - frontend
cd client
npm start
```

The frontend opens in the browser (the default React development address is http://localhost:3000) and communicates with the backend API on the port configured in the `.env` file.

7. API Documentation

The backend exposes a RESTful API that exchanges data in JSON format. Protected endpoints require the header `Authorization: Bearer <token>`. *The endpoints below are derived from the functional requirements in the SRS (sections 3.1 to 3.10); please adjust the paths if your implemented routes differ.*

Access levels: Public = no token needed; Customer, Staff and Admin = valid token with that role (Admin includes all staff rights).

7.1 Authentication and Users (SRS 3.1, 3.2)

Method	Endpoint	Access	Description and parameters
POST	`/api/auth/register`	Public	Register a new user. Body: name, email, phone, password.
POST	`/api/auth/login`	Public	Log in and receive a JWT. Body: email, password.
GET	`/api/users/me`	Any user	View own profile.
PUT	`/api/users/me`	Any user	Update own profile (name, phone).

7.2 Hotels and Rooms (SRS 3.3)

Method	Endpoint	Access	Description and parameters
GET	`/api/hotels`	Public	List hotels.
GET	`/api/hotels/:id`	Public	Get hotel details.
POST	`/api/hotels`	Admin	Add a hotel. Body: name, address, city, description, amenities, images.
PUT	`/api/hotels/:id`	Admin	Edit hotel details.
DELETE	`/api/hotels/:id`	Admin	Delete a hotel.
POST	`/api/hotels/:hotelId/rooms`	Admin	Add a room. Body: roomNumber, type, pricePerNight, capacity, amenities, status.
PUT	`/api/rooms/:id`	Admin	Update a room or change its status.
DELETE	`/api/rooms/:id`	Admin	Delete a room. Returns 409 if the room has active or upcoming bookings.

7.3 Room Search (SRS 3.4)

Method	Endpoint	Access	Description and parameters
GET	`/api/rooms/search`	Public	Search available rooms. Query: city, checkIn, checkOut, guests; optional: minPrice, maxPrice, type, amenities, rating, sort. Returns 400 if checkOut is not after checkIn or checkIn is in the past.
GET	`/api/rooms/:id`	Public	Get room details.

7.4 Bookings (SRS 3.5, 3.6)

Method	Endpoint	Access	Description and parameters
POST	`/api/bookings`	Customer	Create a booking. Body: roomId, checkInDate, checkOutDate, guests. Availability is re-verified; status starts as pending.
GET	`/api/bookings/my`	Customer	List own current, upcoming and past bookings.
GET	`/api/bookings`	Staff, Admin	List all bookings. Query: status, date, guest.
GET	`/api/bookings/:id`	Owner, Staff, Admin	Get booking details.
PUT	`/api/bookings/:id`	Customer	Modify booking dates if the room is available.
PATCH	`/api/bookings/:id/cancel`	Customer, Staff, Admin	Cancel a booking before check-in; the room becomes available again for the freed dates.

7.5 Guests and Check-in / Check-out (SRS 3.7, 3.8)

Method	Endpoint	Access	Description and parameters
POST	`/api/guests`	Staff, Admin	Add a guest record. Body: name, contact, address, idProofType, idProofNumber, bookingId.
GET	`/api/guests`	Staff, Admin	Search guest records. Query: search.
PUT	`/api/guests/:id`	Staff, Admin	Edit a guest record.
PATCH	`/api/bookings/:id/check-in`	Staff, Admin	Check in a guest against a confirmed booking; room status becomes occupied.
PATCH	`/api/bookings/:id/check-out`	Staff, Admin	Check out a guest; booking becomes checked-out and the room is released.

7.6 Payments (SRS 3.9)

Method	Endpoint	Access	Description and parameters
POST	`/api/payments`	Customer	Pay for a booking through the gateway. Body: bookingId, method (card, upi, netbanking).
POST	`/api/payments/offline`	Staff, Admin	Record an offline (cash/counter) payment. Body: bookingId, amount.
GET	`/api/payments`	Staff, Admin	List payments with filters.
GET	`/api/payments/:id/receipt`	Owner, Staff, Admin	Get the payment receipt or invoice.

POST	`/api/payments/:id/refund`	Staff, Admin	Record a full or partial refund for an eligible cancellation.

7.7 Admin Dashboard (SRS 3.10)

Method	Endpoint	Access	Description and parameters
GET	`/api/admin/stats`	Admin	Key metrics: total bookings, active bookings, cancellations, occupancy rate, available rooms and total revenue.
GET	`/api/admin/users`	Admin	List users.
PATCH	`/api/admin/users/:id`	Admin	Change a user's role or activate/deactivate the account.
GET	`/api/admin/reports`	Admin	Export reports. Query: format (csv or pdf).

7.8 Example Requests and Responses

Register a user – `POST /api/auth/register`

```
Request body:
{ "name": "Asha Verma", "email": "asha@example.com",
  "phone": "9876543210", "password": "Secret@123" }

Response 201 Created:
{ "message": "User registered successfully",
  "user": { "_id": "665f1c...", "name": "Asha Verma",
            "email": "asha@example.com", "role": "customer" } }
```

Login – `POST /api/auth/login`

```
Request body:
{ "email": "asha@example.com", "password": "Secret@123" }

Response 200 OK:
{ "token": "<jwt-token>",
  "user": { "_id": "665f1c...", "name": "Asha Verma", "role": "customer" } }

Response 401 Unauthorized (invalid credentials):
{ "message": "Invalid email or password" }
```

Search rooms – `GET /api/rooms/search?city=Bhopal&checkIn=2026-12-10&checkOut=2026-12-12&guests=2`

```
Response 200 OK:
[ { "_id": "6660a2...", "hotelId": "665fe9...", "roomNumber": "204",
    "type": "deluxe", "pricePerNight": 3500, "capacity": 3,
    "amenities": ["WiFi", "AC"], "status": "available" } ]
```

Create a booking – `POST /api/bookings`

```
Request body:
{ "roomId": "6660a2...", "checkInDate": "2026-12-10", "checkOutDate": "2026-12-12",
  "guests": [ { "name": "Asha Verma", "contact": "9876543210" } ] }

Response 201 Created:
{ "_id": "6661b7...", "status": "pending", "totalAmount": 7000 }

Response 409 Conflict (room no longer available):
{ "message": "Room is not available for the selected dates" }
```

The values shown above are sample data used only to illustrate the response format.

8. Authentication

HotelPro uses stateless, token-based authentication with JSON Web Tokens (JWT) and role-based access control (RBAC). No server-side sessions are stored.

- **Registration and passwords:** email must be unique and correctly formatted; passwords must meet minimum strength rules (for example, at least 8 characters) and are stored only as salted bcrypt hashes. Invalid logins return a generic error that does not reveal which field was wrong.
- **Token issue:** after a successful login the server generates a signed JWT containing the user ID and role. The token has a configurable expiry time (for example, 1 hour) and is signed with a secret key kept in an environment variable.
- **Token use:** the client sends the token in the Authorization header using the Bearer scheme for all protected requests. Logout ends the session on the client by discarding the token.
- **Authentication middleware:** verifies the token and rejects missing, expired or tampered tokens with HTTP 401 (Unauthorized).
- **Authorisation middleware:** enforces role-based access and returns HTTP 403 (Forbidden) when the user's role lacks permission.
- **Refresh tokens:** the SRS recommends support for refresh tokens so that sessions can be renewed without forcing a new login (Medium priority).
- **Additional protection:** all communication uses HTTPS/TLS; inputs are validated and sanitised on the server; rate limiting is applied to authentication endpoints to reduce brute-force attempts; CORS is enabled only for approved front-end origins.

Roles and permissions

Role	Permissions
Customer	Register, log in, search rooms, book, pay, view, modify and cancel own bookings.
Staff / Receptionist	Manage guest records, view and filter all bookings, perform check-in and check-out, record offline payments.
Administrator	Full access: manage hotels, rooms, users, bookings and payments; view the dashboard and reports.

9. User Interface

The user interface is a responsive single-page application built with React.js and Bootstrap, and it works on desktop, tablet and mobile screens. The principal screens are described below.

Screen	Description
Home / Landing	Search bar (destination, dates, guests), featured hotels and rooms.
Register / Login	Forms with validation for account creation and sign-in.
Search Results	List or grid of available rooms with filters and sorting.
Room Details	Photos, amenities, price, availability and “Book Now” action.
Booking and Payment	Booking summary, guest details and payment form or redirect.
My Bookings	Customer's booking history with cancel and modify options.
Staff Console	Guest management, bookings list, check-in and check-out.
Admin Dashboard	Statistics, hotel, room and user management, reports.

General interface rules: a consistent navigation bar and footer, a uniform colour theme, visible error and success notifications, loading indicators for asynchronous operations, protected routes that redirect unauthenticated users to the login page, and basic accessibility (readable contrast, labels and keyboard navigation).

Screenshots of these screens are provided in Section 11.

10. Testing

The testing strategy is based on the requirements in the SRS. Test cases are derived from the functional requirements (FR IDs) and the non-functional targets, and each requirement is verified before acceptance.

- **API testing:** every endpoint is tested with Postman, using Postman collections that also serve as API documentation. Checks include request and response format, status codes (200, 201, 400, 401, 403, 404, 409, 500) and role-based access.
- **Functional testing:** end-to-end checks of each module (registration and login, hotel and room management, search, booking, booking management, guest management, check-in and check-out, payment, admin dashboard).
- **Security testing:** invalid, expired and tampered tokens (401); access with an insufficient role (403); input validation against NoSQL injection and XSS; password hashing; rate limiting on authentication endpoints.
- **Performance testing:** pages load within 3 seconds; API responses for login, search and booking return within 2 seconds for 95% of requests; the system supports at least 500 concurrent users under normal load.
- **Compatibility testing:** the latest two versions of Chrome, Firefox, Edge and Safari, and responsive behaviour on desktop, tablet and mobile.
- **Version control:** all changes are tracked with Git and GitHub branches and pull requests.

Key test scenarios

Scenario	Expected Result	Requirement
Register with an email that already exists	Registration rejected with a clear error	FR-REG-02
Access a protected endpoint without a token	HTTP 401 Unauthorized	FR-JWT-04
Customer calls an admin-only endpoint	HTTP 403 Forbidden	FR-JWT-05
Search with check-out date not after check-in date	Search rejected with a validation error	FR-SRC-05
Search for a date range that overlaps an existing booking	That room is not listed	FR-SRC-03
Two users book the same room for the same dates	Only one booking succeeds; the other is rejected	FR-BKG-03
Payment fails	Booking is not confirmed	FR-PAY-04
Delete a room that has upcoming bookings	Deletion prevented	FR-HRM-05
Check in a guest with a confirmed booking	Booking is Checked-in; room is Occupied	FR-CIO-01 , FR-CIO-02
Cancel a booking before check-in	Booking is Cancelled; dates become available again	FR-BMG-02 , FR-BMG-06

11. Screenshots or Demo

Insert the application screenshots in the frames below (one for each principal screen) and add the demo link if available.

[Insert screenshot]	[Insert screenshot]
Home / Landing	**Register / Login**
[Insert screenshot]	[Insert screenshot]
Search Results	**Room Details**
[Insert screenshot]	[Insert screenshot]
Booking and Payment	**My Bookings**
[Insert screenshot]	[Insert screenshot]
Staff Console	**Admin Dashboard**

Demo link: [Add demo link here, if available]

12. Known Issues

No software defects are recorded in the SRS. The following limitations of version 1.0 (from the SRS, "System Constraints and Limitations") should be kept in mind by users and developers:

- The system requires continuous internet access; there is no offline mode.
- Version 1.0 supports a single currency (INR) and a single language (English).
- Payment features depend on the availability and policies of the third-party payment gateway.
- Live availability across simultaneous users depends on request-time checks rather than push notifications.
- MongoDB does not enforce relational integrity, so referential consistency must be handled in the application logic.

- Performance targets assume adequate server and database resources.
- The system does not integrate with external booking portals or property-management hardware such as door-lock systems.
- Native mobile applications, restaurant, spa, housekeeping and inventory modules are outside the scope of version 1.0.

13. Future Enhancements

- **Mobile applications:** native Android and iOS apps or a Progressive Web App.
- **Reviews and ratings:** allow guests to rate and review hotels and rooms.
- **Notifications:** email, SMS and WhatsApp alerts for confirmations, reminders and offers.
- **Dynamic pricing and offers:** seasonal rates, discounts, coupon codes and loyalty programmes.
- **Multi-language and multi-currency support.**
- **Additional services:** restaurant ordering, spa, laundry and housekeeping management.
- **Channel manager integration:** synchronise inventory with online travel agencies such as Booking.com and Expedia.
- **AI-driven features:** personalised recommendations, demand forecasting and chatbot support.
- **Advanced analytics:** revenue forecasting, occupancy trends and downloadable business-intelligence reports.
- **Multi-hotel / multi-tenant support** with separate admin spaces for each property.